

Proiect Sisteme de Prelucrare Grafică

Simularea unei mașini de poliție

Student: Rarinca Robert-Mario
Grupa: 1306A

I. Introducere

Proiectul prezentat a avut ca temă simularea unei mașini de poliție, într-un oraș, în timpul nopții. Timpul simulării (noaptea) a fost ales pentru a demonstra modelele de iluminare și umbrire posibile în OpenGL, în special modelul de iluminare și umbrire phong. Mașina de poliție a fost aleasă datorită semnalelor luminoase speciale. Tema de oras a fost aleasă datorită posibilității surprinderii efectelor de iluminare și umbrire pe clădiri.

Tema generală a proiectului a fost aleasă după ce un coleg a propus să facă "Need For Speed: Most Wanted (2005)" (deși nu cred că am atins același nivel). De la această idee a fost implementată și mașina 'criminală'.

De asemenea, tema a fost inspirată de jocurile cu o temă de stil 'arcade', de aici vine camera foarte îndepărtată de jucător la viteza maximă și înclinările exagerate când se ia un viraj la o viteză mare.

Printre obiectivele propuse se numără: crearea mașinii de poliție, crearea orașului, implementarea modelului phong de iluminare și umbrire, implementarea tehnicilor de "shadow map" și "shadow cube" pentru crearea de umbre, urmărirea unei dezvoltări software ordonată bazată pe principiile programării orientate obiect, precum și respectarea standardelor de scriere programelor C++ (Resource Acquisition Is Initialization – RAII, Rule of Five, Rule of Zero, Minimize RunTime Type Information - RTTI, și altele).

II. Metodologie

La nivel de granularitate mare, pentru dezvoltarea programului de simulare, au fost folosite GLAD, OpenGL 4.6 în mod compatability, GLFW, Visual Studio 2022, compilatorul MSVC (Microsoft Visual C), C++20. Proiectul include de asemenea STB_Image.

La nivel de granularitate medie, s-au folosit tehnici de scriere a unui program orientat obiect (incapsulare, abstractizare, mostenire, polimorfism),

precum și tehnici de scriere în limbajul C++ (Rule of Zero / Three / Five, minimizare RTTI, RAII, etc). S-a optat pentru o structurare a proiectului de tipul “One Class, One File” (O clasă, Un fișier); de fapt este “One Class, Two Files”.

La nivel de granulatitate mică, s-au folosit algoritmi consacrați pentru modelul de iluminare și umbrire phong, dar și pentru tehnicile de umbre de tip “shadow map” și “shadow cube”, cu mici modificări. Pentru crearea orașului, sa folosit o tehnică de “Jittered Grid”, și “Manhattan Routing”. Pentru ‘mașina criminală’ a fost folosit un algoritm Greedy global, și algoritmul BFS (Breadth First Search). Pentru efectul de ‘x-ray’ al mașinii, s-a folosit un alt shader unde s-au setat flag-urile necesare pentru a fi randat ‘deasupra’ celorlalte obiecte. Transparența geamurilor mașinilor a fost făcută prin setarea texturii ferestrelor la o textură de 1x1 pixeli cu o culoare și opacitate dată.

GLAD este un “loader-generator” multi-limbaj pentru mai multe librării grafice (Vulkan, OpenGL, etc). Pentru acest proiect sa folosit pentru a genera un “loader” pentru OpenGL 4.6 in modul compatibilitate.

OpenGL (Open Graphics Library) este o bibliotecă multi-platformă folosit pentru randarea de obiecte 2D și 3D, folosit ca un ‘pod’ standardizat între software și hardware, permitând trimiterea de comenzi direct la GPU.

GLFW este o bibliotecă multi-platforma pentru OpenGL pentru afisare, creare si manipulare de contexte (ferestre) grafice. Acest modul este responsabil pentru desenare si interactiunea cu utilizatorul (preluarea de apasari de taste, preluare miscare mouse, si altele).

Visual Studio 2022 este un “Integrated Development Environment” (prescurtat IDE), care include unelte de compilare, MSVC (Microsoft Visual C / C++), precum și unelte de depanare al codului scris la rulare. Acesta include elemente grafice pentru a ușura dezvoltarea, crearea și depanarea de software.

Standardul oficial (la momentul scrierii) se numește C++20, și este un document publicat de ISO (International Standardization Organisation) ce definește reguli, sintaxa și caracteristici ale limbajului C++. Acest standard este pus la dispoziție, în special pentru dezvoltatorii de compilatoare, pentru a se asigura că codul sursa C++ produce acelasi cod executabil indiferent de compilatorul și hardware-ul (asumând acelasi tip de arhitectura) folosit.

STB_Image este o bibliotecă open-source (cu codul sursa pus la dispoziție) pentru încărcarea de imagini într-un mod mult mai facil într-un cod scris în

limbajul C / C++. Deși STB este o colecție de astfel de librării, pentru acest proiect s-a folosit doar librăria de încărcare de imagini.

Programarea orientată obiect are la bază obiectele, și 4 reguli ce definesc manipularea acestor obiecte. Incapsularea reprezintă ascunderea stării interne a acestor obiecte, și expunerea (sau nu) și manipularea a acestor stări doar prin metode publice. Abstractizarea reprezintă preluarea doar a atributelor necesare și folositoare ale unui obiect când acest este proiectat și implementat, dar și ascunderea detaliilor de implementare a funcționalităților interne. Mostenirea este folosită pentru reutilizarea de cod și crearea de ierarhii de clase. Polimorfismul reprezintă comportamentul diferit al unor obiecte deși au aceeași interfață (de exemplu, o metoda draw() are un comportament diferit pentru un patrat și un cerc, deși ambele sunt figuri geometrice).

În limbajul de programare C++, Rule of Zero / Three / Five este folosit în cazul în care o clasă (nu) gestionează resurse ‘critice’. De obicei aceste resurse critice sunt pointeri ‘raw’ (tip* nume), handler-uri (cum ar fi VBO, EBO, VAO, texture IDs din OpenGL). Rule of Zero este pentru clase ce nu au aceste resurse critice, în special pentru evitarea folosirea acestora fără un motiv întemeiat (cum ar fi pointer la char în loc de std::string). Clasele ce respectă Rule of Zero de obicei au un destructor “default” (generat automat de către compilator). Rule of Three / Five sunt pentru clasele ce au aceste resurse critice, și după regula dată, ar trebui să definească, pe lângă destructor, și constructorul de copiere și operatorul de asignare copiere. Rule of Five este o extindere a Rule of Three, prin definirea constructorului de mutare și operatorului de asignare mutare. Aceste reguli sunt folositoare pentru a evita defectele produse de generarea automată a acestor constructori, operatori și destructor de către compilator, ceea ce poate duce la pierderi de memorie, fenomenul de “double free” (încercarea eliberării unui pointer care a fost deja eliberat) și alte erori la rularea programului.

O clasă ce urmează Rule of Five, ce nu folosește pointer-i, dar handler-uri (campul ID) pentru texturi OpenGL, este următoarea:

```
class Texture
{
private:
    unsigned int ID = 0;
    Texture(const Texture&) = delete;
    Texture& operator=(const Texture&) = delete;
```

```

public:
    Texture(const std::string& path, const
std::string& uniformName);

    Texture(uint8_t* data, int width, int height,
int components, const std::string& uniformName);

    Texture(Texture&& other) noexcept;
    ~Texture();

    // Alte metode specifice.

    Texture& operator=(Texture&& other) noexcept;
};

Texture& Texture::operator=(Texture&& other) noexcept
{
    if (this == &other)
    {
        return *this;
    }

    glDeleteTextures(1, &ID);
    ID = std::exchange(other.ID, 0);
    uniformName = std::move(other.uniformName);
    return *this;
}

Texture::Texture(Texture&& other) noexcept :
    ID(std::exchange(other.ID, 0)),
    uniformName(std::move(other.uniformName)) {}

Texture::~~Texture() { glDeleteTextures(1, &ID); }

```

RAII (Resource Acquisition Is Initialization) este o paradigmă limbajului de programare C++. Acesta stipulează că achiziționarea de resurse (de obicei aceleași ‘resurse critice’ ca mai sus; pointeri, handlers, etc) necesită instanțierea unui obiect, de obicei de tip ‘wrapper’, peste aceste resurse. Această regulă face ca durata de viața acestui obiect, și implicit a acestor resurse critice alocate, să fie strâns legate. În acest sens, de obicei se spune că

inițializarea obiectului alocă resursa, și distrugerea obiectului eliberează resursa.

Un exemplu de RAII ar fi următorul (clasa Shader este un wrapper peste shader-urile specifice OpenGL)

```
auto phong =
std::make_shared<Shader>("phong.vert", "phong.frag");

shaders[static_cast<size_t>(ShaderType::Phong)] =
phong;

auto depth =
std::make_shared<Shader>("shadow_map.vert",
"shadow_map.frag");

shaders[static_cast<size_t>(ShaderType::ShadowMap)]
= std::move(depth);

auto cube =
std::make_shared<Shader>("shadow_cube.vert",
"shadow_cube.frag", "shadow_cube.geom");

shaders[static_cast<size_t>(ShaderType::ShadowCube)]
= std::move(cube);

auto xray = std::make_shared<Shader>("xray.vert",
"xray.frag");

shaders[static_cast<size_t>(ShaderType::Xray)] =
std::move(xray);
```

RTTI (RunTime Type Information) este o caracteristică a limbajului de programare C++. Aceasta este folosită pentru a prelua tipul unei variabile (de obicei o clasă) în timpul rulării, pentru a face cast, sau operații specifice pe acea variabilă. În cele mai multe ghiduri de scriere a unui program C++, se recomandă a nu se folosi acest mecanism, și înlocuirea sa cu alte metode de apelare acestor funcționalități (sablonul Visitor, re-proiectarea ierarhiei de clase pentru a ține cont de această funcționalitate, și altele).

Structurarea proiectului în modul “One Class, One File” (de fapt, “One Class, Two Files”) înseamnă, după cum spune și numele, că fiecare clasă este definită într-un singur fișier (de obicei cu extensia .h sau .hpp), iar implementarea propriu-zisă într-un alt fișier (de obicei cu extensia .c sau .cpp). Denumirea de “One Class, One File” este mult mai răspândită decât “One Class, Two Files”, deși în acest caz se folosesc două fișiere pentru definire și

implementare. Acest mod de structurare este în contrast cu stilul “vechi” de structurare monolit (un singur fișier .h cu toate declarațiile, clasele, etc, și un singur fișier .cpp cu implementările).

Modelul de iluminare și umbrire phong este caracterizat de trei tipuri de iluminare, cea ambientă (constantă), difuză (lumina ce lovește obiectul la un unghi) și speculara (partea strălucitoare a unui obiect). Acestea se calculează preluând distanța de la obiect la sursa de lumina (mai puțin cea ambientă, care este constantă indiferent de distanță), și folosind acestea, calcularea de coeficienți pentru o anumită lumină în scenă cu o putere de iluminare dată, ceea ce va face ca obiectul să pară iluminat într-un mod natural de către sursa de lumină.

În shader-ul de fragmente al modelului de iluminare și umbrire phong, se definește următoarea buclă pentru tratarea luminii provenite de la ‘point lights’ (lumini omnidirecționale):

```
for (int i = 0; i < pointLightCount; i++)
{
    if(pointLights[i].intensity <= 0.001f)
    {
        continue;
    }

    float dist = length(pointLights[i].position -
FragPos);
    if (dist > 30.0)
    {
        continue;
    }

    float currentFarPlane = (i < 2) ?
lightBarFarPlane : blinkerFarPlane;

    float window = 1.0;

    float farFade = clamp((currentFarPlane - dist) /
window, 0.0, 1.0);
```

```

        float attenuation = (1.0 /
(pointLights[i].constant + pointLights[i].linear * dist
+ pointLights[i].quadratic * (dist * dist))) * farFade;

        if (attenuation < 0.001)
        {
            continue;
        }

        vec3 lightDir = normalize(pointLights[i].position
- FragPos);

        float NdotL = max(dot(norm, lightDir), 0.0);
        if (NdotL <= 0.0)
        {
            continue;
        }

        float shadow = 0.0;

        vec3 reflectDir = reflect(-lightDir, norm);

        float spec = pow(max(dot(viewDir, reflectDir),
0.0), shininess);

        vec3 ambient = (ambientStrength * 0.05) *
pointLights[i].color;

        vec3 diffuse = NdotL * pointLights[i].color *
pointLights[i].intensity;

        vec3 specular = specularStrength * spec *
pointLights[i].color * pointLights[i].intensity;

        result += (ambient + (1.0 - shadow) * (diffuse +
specular)) * attenuation;
    }

```

Technica de umbre de tipul “shadow mapping” folosește metoda de ‘depth testing’ din perspectiva luminii (~ camera privește în direcția luminii). Aceasta randează scena din perspectiva luminii și calculează distanța (‘depth’) a unui pixel în prima parte, iar în randarea principală (obiecte, culori, etc), poziția aceluiasi pixel este comparată cu ‘depth map’-ul (textura) umbrei. Dacă pixel-ul se află ‘mai departe’ de poziția pixelului în textura umbreii, acesta este

ascuns și desenat ca o umbră. Această tehnică este potrivită pentru lumini de tip reflector (ce iluminează într-o singură direcție; ‘spotlight’).

Pentru calcularea umbrei date de un shadow map, sa folosit următoarea implementare:

```
float ShadowCalculation(sampler2D shadowMap, vec4
fragPosLightSpace, float NdotL, vec3 normal)
{
    vec3 projCoords = fragPosLightSpace.xyz /
fragPosLightSpace.w;
    projCoords = projCoords * 0.5 + 0.5; // [-1,1] ->
[0,1]
    if(projCoords.z > 1.0)
    {
        return 0.0;
    }
    if(projCoords.x < 0.0 || projCoords.x > 1.0 ||
projCoords.y < 0.0 || projCoords.y > 1.0)
    {
        return 0.0;
    }
    float offsetScale = 0.05;
    vec2 offset = normal.xz * (1.0 - NdotL) *
offsetScale * (1.0 / textureSize(shadowMap, 0));
    float bias = max(0.0003 * (1.0 - NdotL),
0.0003);
    float currentDepth = projCoords.z;
    vec2 texelSize = 1.0 / textureSize(shadowMap, 0);
    float shadow = 0.0;
    for(int x = -1; x <= 1; ++x)
    {
        for(int y = -1; y <= 1; ++y)
```

```

    {
        float pcfDepth = texture(shadowMap,
projCoords.xy + offset + vec2(x, y) * (texelSize *
2.0)).r;

        shadow += currentDepth - bias > pcfDepth
? 1.0 : 0.0;
    }
}

return shadow / 9.0;
}

```

Technica de umbre de tipul “shadow cube” este o generalizare a metodei de “shadow mapping”, partea de cub venind de la faptul că lumina produce un ‘shadow map’ în fiecare față a cubului (sus, jos, stânga, dreapta, față, spate). Această tehnică este potrivită pentru lumini omnidirecționale (cum ar fi un bec). Această tehnică, pentru a evita calcule matematice grele, și pentru evitarea zgomotului alb, folosește o metodă popularizată de Jorge Jimene (Interweave Gradient Noise - IGN) pentru a dispersa pixeli într-un mod ce pare aleator, dar menține o consistență vizuală, dar datorită acestei metode, lumina, privită de aproape, tot prezintă un zgomot mic, dar mai bun decât dacă s-ar fi aplicat un ‘zgomot alb’. O altă metodă de a face umbrele date de aceasta tehnică de “shadow cube” este folosirea unor “sample disks”, cu o distribuție Poissoniană, pentru ‘spargerea’ de structuri repetitive din umbră.

Pentru generarea de umbre conform tehnicii ‘shadow cube’ s-au folosit următoarele:

```

const vec3 sampleDisk[8] = vec3[(
    vec3(0.5238, 0.3411, 0.4110), vec3(-0.2411, 0.5620,
-0.1566),
    vec3(-0.1566, -0.4828, 0.4021), vec3(0.5419, -0.2041,
-0.4411),
    vec3(0.1212, 0.1212, -0.9000), vec3(-0.9000, 0.1212,
0.1212),
    vec3(0.1212, -0.9000, 0.1212), vec3(0.0, 0.0, 0.0)
);

float InterleavedGradientNoise(vec2 screenPos)

```

```

{
    vec3 magic = vec3(0.06711056, 0.00583715,
52.9829189);

    return fract(magic.z * fract(dot(screenPos,
magic.xy)));
}

vec2 rotate(vec2 v, float a)
{
    float s = sin(a);
    float c = cos(a);
    return vec2(v.x * c - v.y * s, v.x * s + v.y *
c);
}

float PointShadowCalculation(vec3 fragPos, vec3
lightPos, samplerCube depthCube, float farPlane)
{
    vec3 fragToLight = fragPos - lightPos;
    float currentDepth = length(fragToLight);
    float spread = (currentDepth / farPlane) * 0.15;
    float rotation =
InterleavedGradientNoise(gl_FragCoord.xy) * 6.2831;
    float shadow = 0.0;
    float bias = 0.05;
    for(int i = 0; i < 8; ++i)
    {
        vec2 rotatedXZ = rotate(sampleDisk[i].xz,
rotation);
        vec3 jitteredDir = vec3(rotatedXZ.x,
sampleDisk[i].y, rotatedXZ.y);
        float closestDepth = texture(depthCube,
fragToLight + jitteredDir * spread).r;

```

```

        closestDepth *= farPlane;
        if(currentDepth - bias > closestDepth)
        {
            shadow += 1.0;
        }
    }
    return shadow / 8.0;
}

```

Orașul este creat mai întâi generând intersecțiile (~ semințele), aplicând un offset aleator (“jittered grid”) pentru a evita liniile drepte. După aceasta este folosit un algoritm de ‘grid ray tracing’ (“Manhattan Routing”) pentru a genera conexiunile (drumurile) dintre aceste intersecții. Acest algoritm preia drumul cel mai scurt în linie dreaptă dintre puncte.

Algoritmul este urmatorul:

```

    std::vector<std::vector<CityTileType>>
CityModel::GenerateCity(const std::weak_ptr<Shader>&
shader, int size)
{
    std::vector<std::vector<CityTileType>>
matrix(size, std::vector<CityTileType>(size,
CityTileType::Skyscraper));

    struct Node
    {
        int x;
        int z;
    };

    std::vector<std::vector<Node>> grid;
    int spacing = (size <= 12) ? 4 : 4;
    for (int z = 1; z < size; z += spacing)
    {
        std::vector<Node> row;
    }
}

```

```

        for (int x = 1; x < size; x += spacing)
        {
            int ox = (rand() % 2);
            int oz = (rand() % 2);

            row.push_back({ std::clamp(x + ox, 0,
size - 1), std::clamp(z + oz, 0, size - 1) });
        }
        grid.push_back(row);
    }

    auto DrawSimplePath = [&](Node n1, Node n2) {
        int cx = n1.x, cz = n1.z;
        while (cx != n2.x)
        {
            matrix[cz][cx] =
CityTileType::Intersection;
            cx += (n2.x > cx) ? 1 : -1;
        }
        while (cz != n2.z)
        {
            matrix[cz][cx] =
CityTileType::Intersection;
            cz += (n2.z > cz) ? 1 : -1;
        }
        matrix[cz][cx] = CityTileType::Intersection;
    };

    for (size_t r = 0; r < grid.size(); ++r)
    {
        for (size_t c = 0; c < grid[r].size(); ++c)
        {

```

```

        if (c + 1 < grid[r].size())
        {
            DrawSimplePath(grid[r][c], grid[r][c
+ 1]);
        }
        if (r + 1 < grid.size())
        {
            DrawSimplePath(grid[r][c], grid[r +
1][c]);
        }
    }
}

for (int z = 0; z < size; ++z)
{
    for (int x = 0; x < size; ++x)
    {
        if (matrix[z][x] ==
CityTileType::Skyscraper)
        {
            if ((rand() % 100) < 15)
            {
                matrix[z][x] =
CityTileType::Park;
            }
        }
    }
}

auto& finalMatrix = matrix;

auto IsRoad = [&](int r, int c) {
    if (r < 0 || r >= size || c < 0 || c >= size)

```

```

        {
            return false;
        }

        CityTileType t = matrix[r][c];

        return t != CityTileType::Skyscraper && t !=
CityTileType::Park;
    };

    for (int z = 0; z < size; ++z)
    {
        for (int x = 0; x < size; ++x)
        {
            if (matrix[z][x] ==
CityTileType::Skyscraper || matrix[z][x] ==
CityTileType::Park)
            {
                continue;
            }

            bool N = IsRoad(z - 1, x), S = IsRoad(z +
1, x),

                E = IsRoad(z, x + 1), W = IsRoad(z, x
- 1);

            int conn = static_cast<int>(N) +
static_cast<int>(S) + static_cast<int>(E) +
static_cast<int>(W);

            if (conn >= 3)    finalMatrix[z][x] =
CityTileType::Intersection;

            else if (N && S)  finalMatrix[z][x] =
CityTileType::StreetVertical;

            else if (E && W)  finalMatrix[z][x] =
CityTileType::StreetHorizontal;

            else if (S && E)  finalMatrix[z][x] =
CityTileType::CornerNE;

```

```

        else if (S && W)    finalMatrix[z][x] =
CityTileType::CornerNW;

        else if (N && E)    finalMatrix[z][x] =
CityTileType::CornerSE;

        else if (N && W)    finalMatrix[z][x] =
CityTileType::CornerSW;

        else                finalMatrix[z][x] =
CityTileType::Intersection;

    }

}

BuildModelFromMatrix(shader, finalMatrix);

Bake(shader);

return finalMatrix;

}

```

Pentru algoritmul ‘mașinii criminale’ s-a folosit o parcurgere de tip Greedy globală. În esență, parcurge toate dalele (tile-urile) orașului, și o găsește pe cea cu cea mai mare distanță față de utilizator, prioritizând mersul în aceeași direcție în care deja se deplasează, evitând astfel întoarcerile bruște care ar strica experiența simulatorului. Pentru parcurgerea efectivă a drumului până la acea destinație, s-a optat pentru algoritmul BFS pentru găsirea drumului, și implicit a nodurilor ce trebuie traversate, fiind un algoritm decent pentru dimensiunea hărții date, deși acesta poate fi înlocuit cu algoritmul A* în dezvoltări ulterioare.

Algoritmii folosiți sunt următorii:

```

glm::ivec2
CivilianCarActive::FindFleeTarget(glm::ivec2
playerGrid, const
std::vector<std::vector<CityTileType>>& grid)
{
    glm::ivec2 bestTile = { 0, 0 };
    float bestScore = -1000000.0f;

    glm::vec2 currentPosGrid =
glm::vec2(WorldToGrid(transform.position));

```

```

        glm::vec2
currentDir(sin(glm::radians(currentAngle)),
cos(glm::radians(currentAngle)));

    for (int y = 0; y < 25; ++y)
    {
        for (int x = 0; x < 25; ++x)
        {
            if (grid[y][x] != CityTileType::Park &&
grid[y][x] != CityTileType::Skyscraper)
            {
                glm::vec2 tilePos(x, y);

                float distToPlayer =
glm::distance(tilePos, glm::vec2(playerGrid));

                glm::vec2 toTileVec = tilePos -
currentPosGrid;

                float distToTile =
glm::length(toTileVec);

                if (distToTile < 0.1f)
                {
                    continue;
                }

                glm::vec2 dirToTile = toTileVec /
distToTile;

                float alignment =
glm::dot(currentDir, dirToTile);

                float alignmentMultiplier =
(alignment <= 0.0f) ? 0.2f : 1.0f;

                float score = (distToPlayer *
alignmentMultiplier) + (alignment * 5.0f);

                if (score > bestScore)
                {
                    bestScore = score;

```

```

        bestTile = { x, y };

    }

}

}

return bestTile;

}

std::vector<glm::ivec2>
CivilianCarActive::CalculatePath(glm::ivec2 start,
glm::ivec2 target, const
std::vector<std::vector<CityTileType>>& grid)
{
    if (start == target) return {};
    std::queue<glm::ivec2> frontier;
    frontier.push(start);
    std::unordered_map<int, glm::ivec2> cameFrom;
    cameFrom[start.x << 16 | start.y] = start;
    bool found = false;
    std::vector<glm::ivec2> neighbors = { {0,1}, {0,-
1}, {1,0}, {-1,0} };
    while (!frontier.empty())
    {
        glm::ivec2 current = frontier.front();
        frontier.pop();
        if (current == target) {
            found = true;
            break;
        }
        for (const auto& offset : neighbors)
        {

```

```

        glm::ivec2 next = current + offset;
        if (next.x >= 0 && next.x < 25 && next.y
            >= 0 && next.y < 25)
        {
            int nextKey = next.x << 16 | next.y;
            if ((grid[next.y][next.x] !=
CityTileType::Park && grid[next.y][next.x] !=
CityTileType::Skyscraper) && cameFrom.find(nextKey) ==
cameFrom.end())
            {
                frontier.push(next);
                cameFrom[nextKey] = current;
            }
        }
    }

    std::vector<glm::ivec2> finalPath;
    if (found)
    {
        glm::ivec2 curr = target;
        while (curr != start)
        {
            finalPath.push_back(curr);
            curr = cameFrom[curr.x << 16 | curr.y];
        }
        std::reverse(finalPath.begin(),
finalPath.end());
    }

    return finalPath;
}

```

Efectul de ‘xray’ când ‘mașina criminală’ se depărtează este realizat print-un shader simplu, cel de fragment desenând o singură culoare dată cu o anumită opacitate. Opacitatea este calculată în funcție de distanță. Pentru că acei pixeli să fie vizualizați deasupra celorlalți pixeli, sa folosit funcția `glDepthFunc(GL_GREATER)`.

```
auto& xrayS =
shaders[static_cast<size_t>(ShaderType::Xray)];

    float distance = glm::distance(mainCar-
>GetPosition(), chaseCar->GetPosition());

    if (distance > 50.0f)
    {

        xrayS->UseShader();

        glEnable(GL_BLEND);

        glBlendFunc(GL_SRC_ALPHA,
GL_ONE_MINUS_SRC_ALPHA);

        glDepthMask(GL_FALSE);

        glDepthFunc(GL_GREATER);

        glUniformMatrix4fv(xrayS-
>GetUniformLocation("projViewMatrix"), 1, GL_FALSE,
&projViewMatrix[0][0]);

        float alpha = glm::clamp((distance -
15.0f) / 100.0f, 0.2f, 0.7f);

        glUniform3f(xrayS-
>GetUniformLocation("glowColor"), 1.0f, 0.4f, 0.0f);

        glUniform1f(xrayS-
>GetUniformLocation("opacity"), alpha);

        chaseCar->Draw(xrayS);

        glDepthFunc(GL_LESS);

        glDepthMask(GL_TRUE);

        glDisable(GL_BLEND);

    }
```

Pentru a ne folosi de texturare, pentru opacitatea ferestrelor mașinilor sa folosit un așa-zis “programming hack”. În fragment shader-ul phong, culoarea

este doar un vector de 3 elemente (RGB), dar textura este un (RGBA), așa că transparența ferestrelor sa realizat prin setarea texturii acestora la o textură de 1x1 pixeli cu o culoare și o transparență dată.

```
if (!data->windowTex)
{
    data->windowTex = Texture::CreateSolidColor(
        static_cast<unsigned char>(color.r * 255),
        static_cast<unsigned char>(color.g * 255),
        static_cast<unsigned char>(color.b * 255),
        alpha,
        "texture_diffuse"
    );
}

std::unique_ptr<Mesh> mesh = nullptr;

switch (type)
{
case WindowType::Triangle:
{
    mesh = std::make_unique<WedgeMesh>(shader,
    Transform{ .position = pos, .rotation = rot, .scale =
scale }, color);
    break;
}
case WindowType::Flat:
default:
{
    mesh = std::make_unique<CubeMesh>(shader,
    Transform{ .position = pos, .rotation = rot, .scale =
scale }, color);
    break;
}
```

```

    }

    }

    CivilianCarPart& part = CreatePart(std::move(mesh),
CivilianCarMeshType::Window,
CivilianCarPartIndex::None, pos);

    }

    Texture::Texture(uint8_t* data, int width, int
height, int components, const std::string& uniformName)
:
    uniformName(uniformName)
    {
        glGenTextures(1, &ID);
        glBindTexture(GL_TEXTURE_2D, ID);
        glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
        GLenum format = (components == 4) ? GL_RGBA :
GL_RGB;
        glTexImage2D(GL_TEXTURE_2D, 0, format, width,
height, 0, format, GL_UNSIGNED_BYTE, data);
        glTexParameteri(GL_TEXTURE_2D,
GL_TEXTURE_MIN_FILTER, GL_LINEAR);
        glTexParameteri(GL_TEXTURE_2D,
GL_TEXTURE_MAG_FILTER, GL_LINEAR);
    }

    std::shared_ptr<Texture>
Texture::CreateSolidColor(uint8_t r, uint8_t g, uint8_t
b, uint8_t a, const std::string& uniformName)
    {
        uint8_t data[] = { r, g, b, a };
        return std::make_shared<Texture>(data, 1, 1, 4,
uniformName);
    }

```

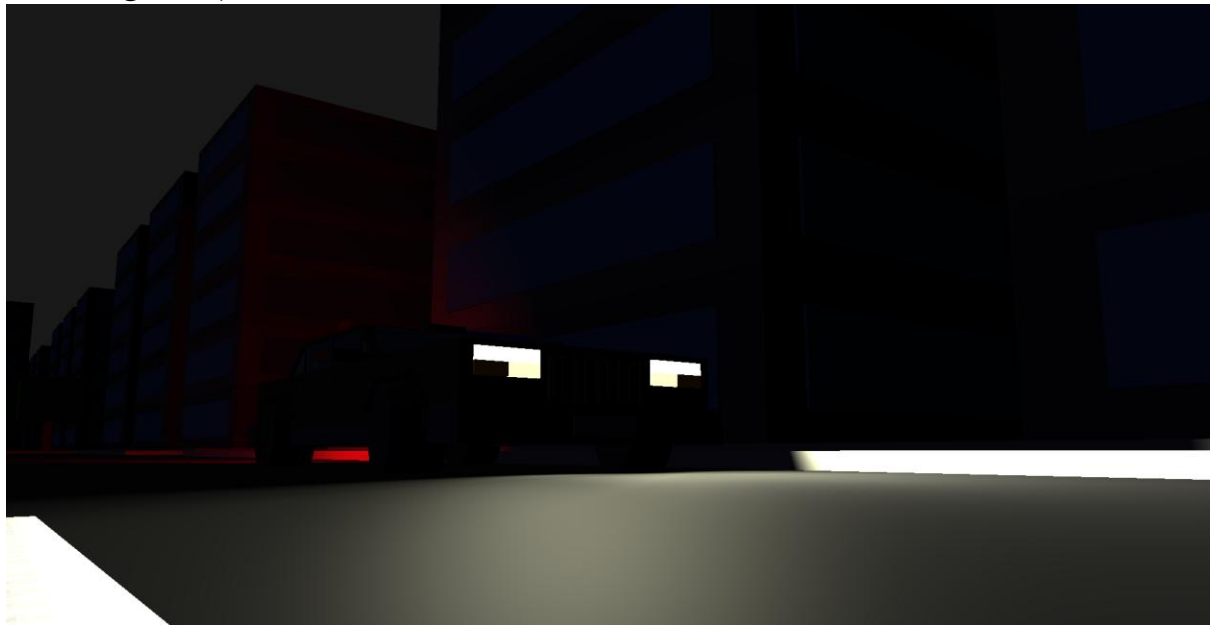
III. Rezultate

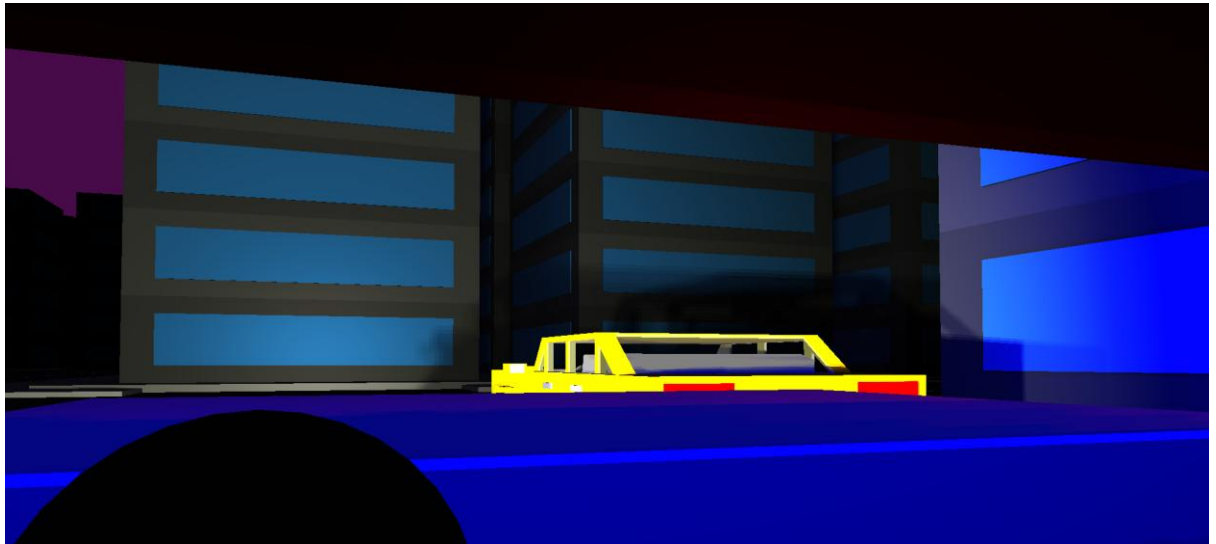
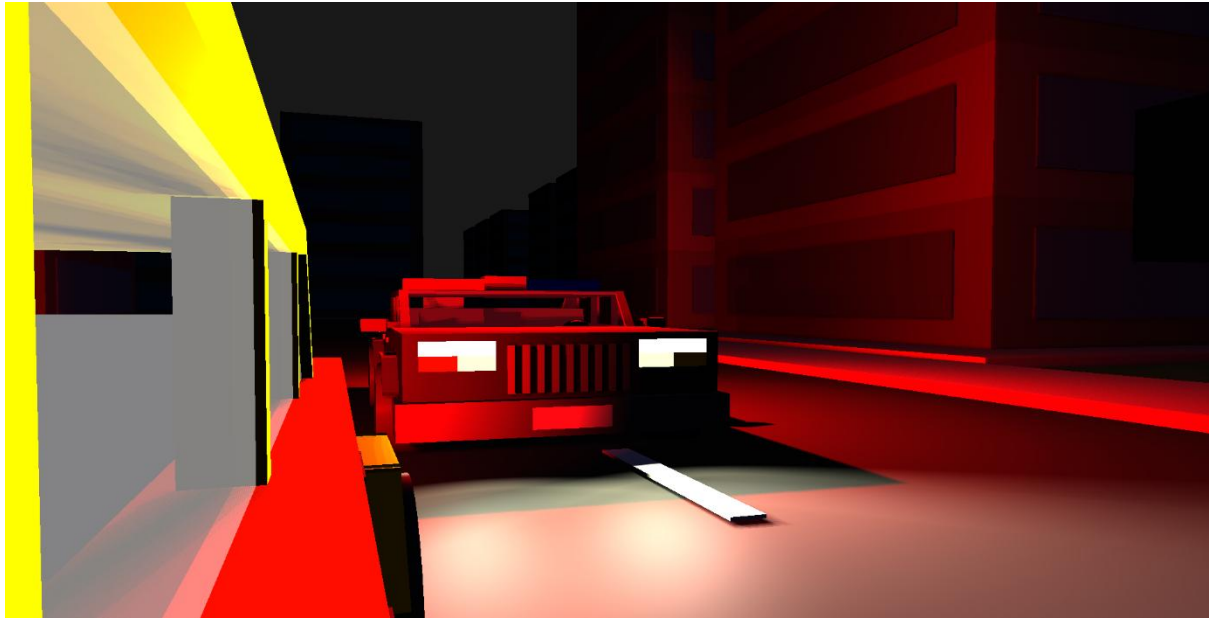
În urma multor cicluri de dezvoltare, s-a ajuns la un demo acceptabil al simulatorului. Acesta include mașina de poliție controlată de utilizator, posibilitatea de a stinge sau aprinde luminile ale respectivei mașini, crearea orașului, 'mașinii criminale' și algoritmilor aferenți, metodei de iluminare și umbrire phong, tehnicilor de umbre 'shadow mapping' și 'shadow cube'. Pe lângă acestea, s-a implementat și o modalitate de a instanția oricâte mașini de poliție statice, deși această funcționalitate nu este folosită în simulator încă.

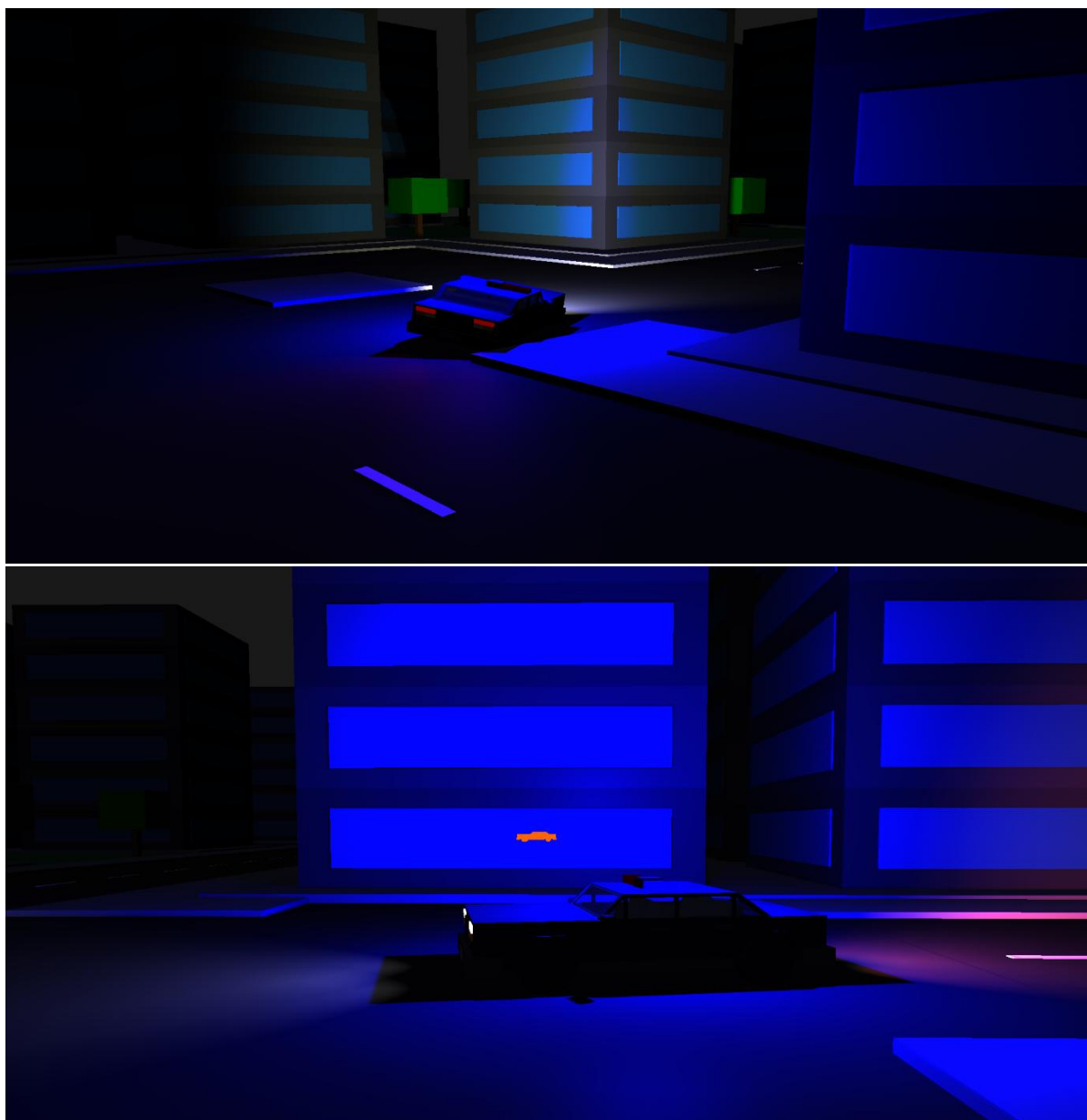
Efectele de umbră sunt satisfăcătoare pentru demo-ul dat, fiind create într-un mod ce evită efectul de "moire" ("shadow acne"), și cu o decuplare de obiect ("peter panning") minimă.

Efectul de X-RAY este implementat corect, dar poate dispărea dacă 'mașina criminală' trece de 'far plane'-ul camerei (~ 200 unități).

O altă funcționalitate deosebit de importantă este tratarea reală a accelerației, vitezei și decelerației mașinii, deși nu la un nivel foarte avansat, are un comportament de așteptat, dar în acest sens, schimbările de viteză și direcția afectează rotația mașinii într-un mod natural (datorită inerției, forței centrifuge, etc).







IV. Concluzii

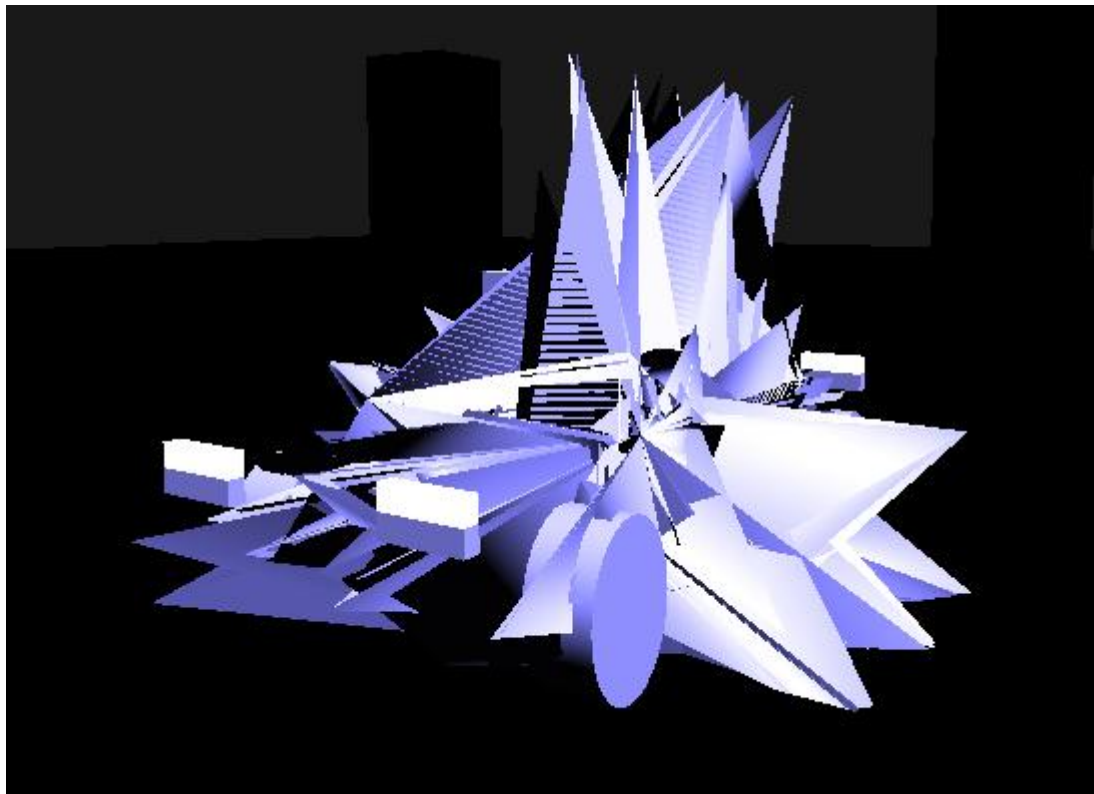
În acest proiect sa implementat un simulator al unei mașini de poliție, într-un oraș, în timpul nopții. Utilizatorul poate conduce mașina și controla luminile. Simulatorul s-a concentrat pe partea vizuală, având implementate modelul de iluminare și umbrire phong, precum și tehnicile de umbre “shadow map” și “shadow cube”.

Eliminarea efectului de ‘moire’ (‘shadow acne’) în timp ce se încerca să se minimizeze efectul de decuplare a umbrelor (‘peter panning’) a reprezentat un punct de dificultate mărită. Un alt punct dificil a fost găsirea unei balanțe între fidelitate grafică și performanță. Performanța curenta a

simulatorului este acceptabila, dar pentru orice alte dezvoltări ulterioare, necesită optimizări majore, deși fidelitatea grafica curentă este la un nivel satisfăcător.

O direcție de dezvoltare este restructurarea responsabilităților claselor mai mari (Application.h, PoliceCarModel.h, CivilianCarActive.h), prin spargerea acestora în alte clase. Pe partea de mecanici ale simulatorului se pot adăuga coliziuni cu mediul, dar și un sistem de fizică mai avansat, ceea ce ar face posibilă adăugarea funcționalității de suspensii ale mașinilor.

Contribuția personală a fost proiectarea cât mai balansată între viteza și corectitudine, precum și separarea responsabilităților, dar și expandarea ideii originale propuse.



V. Referințe Bibliografice

Interweaved Gradient Noise -

<https://blog.demofox.org/2022/01/01/interleaved-gradient-noise-a-different-kind-of-low-discrepancy-sequence/>

Learn Open GL (model de iluminare și umbrire phong, tehnicile de shadow mapping, shadow cube, texturare, camera, meshes, shaders) -

<https://learnopengl.com/>

Rule of Three / Five / Zero -

https://en.cppreference.com/cpp/language/rule_of_three

GLFW - <https://github.com/glhf/glhf>

GLAD - <https://github.com/dav1dde/glad>

RAII - <https://en.cppreference.com/cpp/language/raii>

OpenGL wiki - https://wikis.khronos.org/opengl/Main_Page

How to only get one key press on GLFW -

<https://stackoverflow.com/questions/51873906/is-there-a-way-to-process-only-one-input-event-after-a-key-is-pressed-using-glfw>